

**МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ**

ОТЧЁТ
по лабораторной работе №1
по дисциплине «Основы машинного обучения»
Тема: ПРЕДОБРАБОТКА ДАННЫХ ДЛЯ МАШИННОГО
ОБУЧЕНИЯ

Студент гр. 3312	_____	Барченков П.А.
Студент гр. 3312	_____	Лебедев И.А.
Студент гр. 3312	_____	Шарапов И.Д.
Преподаватель	_____	Петруша П.Г.

Санкт-Петербург
2025

Содержание

ЦЕЛЬ РАБОТЫ	3
ОПИСАНИЕ ЗАДАНИЯ	3
1. ПОДГОТОВКА ДАННЫХ.....	4
1.1. Анализ столбцов.....	4
1.2. Работа с данными	6
2. ВИЗУАЛИЗАЦИЯ ДАННЫХ.....	7
2.1. Диаграммы рассеяния.....	7
2.2. Ящики с усами.....	9
2.3. Гистограммы.....	12
3. СТАТИСТИЧЕСКИЙ АНАЛИЗ	14
ВЫВОДЫ.....	17
ПРИЛОЖЕНИЕ	19

ЦЕЛЬ РАБОТЫ

Получение и закрепление навыков предобработки большого количества данных для дальнейшего применения методов машинного обучения.

ОПИСАНИЕ ЗАДАНИЯ

Самостоятельно выбрать набор сырых (не разобранных) данных на сайте kaggle.com по следующим критериям:

- Число столбцов признаков – не менее 10;
- Число записей – не менее 10000;
- Набор данных имеет пропуски.

Очистить данные (удалить пропуски, нормализовать данные, удалить дубликаты).

Визуализировать значимые признаки с помощью таких методов как:

- Диаграммы рассеяния
- Ящики с усами
- Гистограммы

Проанализировать корреляцию данных, составив матрицу корреляций.

1. ПОДГОТОВКА ДАННЫХ

В качестве датасета нами был выбран <https://www.kaggle.com/datasets/ikramshah512/amazon-products-sales-dataset-42k-items-2025>. В нём собрано более 42 тыс. записей о покупках на сайте Amazon. Имеется 16 столбцов с количественными и качественными данными, что позволит нам выбрать наиболее подходящие для анализа.

1.1. Анализ столбцов

На этом этапе были изучено назначение каждого столбца и выбрана стратегия по его обработке:

- ***title*** – полное название товара. Привели текст к нижнему регистру, для простого детектирования дубликатов.
- ***rating*** – средний рейтинг покупателей по пятибалльной шкале. Преобразовали в число с плавающей точкой. Заполнили пропуски медианным значением.
- ***number_of_reviews*** – общее количество отзывов покупателей. Преобразовали в число с плавающей точкой. Заполнили пропуски медианным значением.

Так как признаки ***rating*** и ***number_of_reviews*** в отдельности не имеют большого смысла (товар с высоким рейтингом, но маленьким количеством отзывов не обязательно хороший), то объединили два этих столбца.

Для этого создали столбец ***reputation*** (репутация). Его значение равно среднему арифметическое нормализованных значений ***rating*** и ***number_of_reviews*** для этого товара. Столбцы ***rating*** и ***number_of_reviews*** удалили.

- ***bought_in_last_month*** – количество единиц, купленных за последний месяц. Изначально представляет из себя текстовое значение, преобразовали в натуральное число. После нормализации сгруппировали в три категории спроса:

- «*low*», где нормализованное значение от 0 до 0.01

- «*middle*», где нормализованное значение от 0.01 до 0.1
- «*high*», где нормализованное значение от 0.1 до 1

Полученные группы записали в новый столбец ***demand*** (спрос). Столбец ***bought_in_last_month*** удалили.

- ***current/discounted_price*** – текущая цена после скидки. Является числом с плавающей точкой, просто переименовали столбец в ***total_price*** для удобства. Пропуски заполнили медианным значением.

- ***price_on_variant*** – столбец имеет странные данные (не всегда это число). Не представляет интереса для анализа, поэтому был удалён.

- ***listed_price*** – изначальная цена. В столбце указана либо цена в долларах, либо текст «*No Discount*». Если в строке указан текст, на его место поставили значение из ***total_price***.

- ***is_best_seller*** – указывает, помечен ли товар как «Бестселлер». Не представляет интереса для анализа, поэтому был удалён.

- ***is_sponsored*** – является ли товар рекламным или попал в рекомендации естественным образом. Имеется два значения «*Organic*» и «*Sponsored*». Заменяли соответственно на 0 и 1.

- ***is_couponed*** – наличие специальных скидочных купонов. Встречаются строки трёх типов:

1. *No Coupon*
2. *Save \$XX.XX with coupon*
3. *Save XX% with coupon*

Заменяли их на пять категорий: «*no*», «*>=10\$*», «*<10\$*», «*>=25%*», «*<25%*».

- ***buy_box_availability*** – наличие кнопки BuyBox на странице поиска Amazon. Строки «*Add to cart*» заменили на 1, остальные на 0.

- ***delivery_details*** – ожидаемая дата доставки. Зависит от даты заказа, которой у нас нет, поэтому невозможно проследить зависимость. Столбец удалили.

- ***sustainability_badges*** – теги, связанные с экологичностью и устойчивым развитием. Данные качественные, слишком много пропусков, заполнить их невозможно. Удалили столбец.

- ***image_url*** – прямая ссылка на изображение товара. Не представляет интереса для анализа, поэтому был удалён.

- ***product_url*** – официальная страница товара на Amazon. Не представляет интереса для анализа, поэтому был удалён.

- ***collected_at*** – дата, когда были собраны данные. Не представляет интереса для анализа, поэтому был удалён.

1.2. Работа с данными

После анализа столбцов, последовательно выполнили соответствующие преобразования. Нормализовали данные с помощью функции `MinMaxScaler()`.

С помощью функции `data.duplicated().sum()` выяснили, что в таблице имеется 33413 дубликатов. Удалили их с помощью функции `data.drop_duplicates()`. В итоге в таблице осталось 9262 строки. Большое количество дубликатов вполне ожидаемо в случае автоматической выгрузки с маркетплейса. Оставшихся данных достаточно для качественного анализа.

2. ВИЗУАЛИЗАЦИЯ ДАННЫХ

На этом этапе мы с помощью инструментов Python создали диаграммы по очищенным данным, проанализировали их и сделали выводы.

2.1. Диаграммы рассеяния

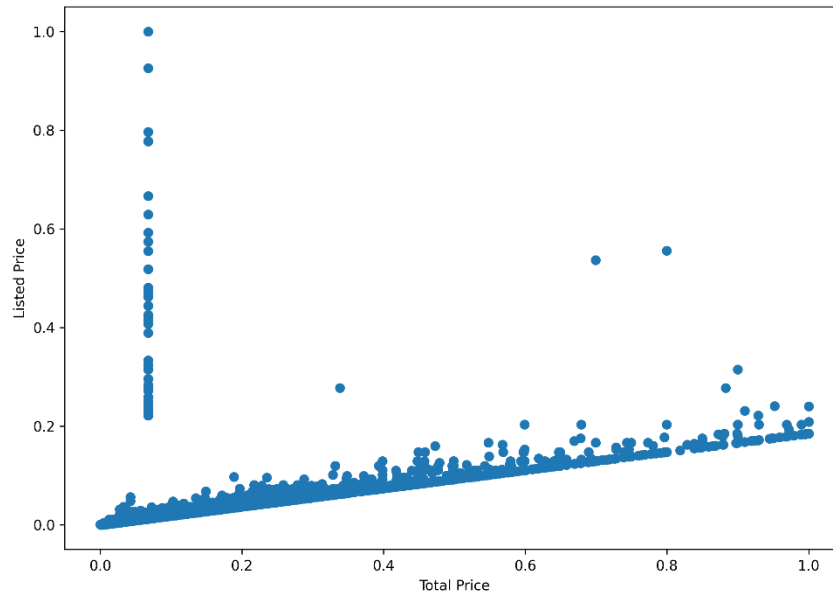


Рисунок 1 – Диаграмма рассеяния `listed_price` по `total_price`

Наблюдается почти линейная зависимость (рис. 1): обе оси – это базовая и текущая цена одного и того же товара. В коде строки `listed_price == "No Discount"` заменяются значением из `total_price`, поэтому значительная часть точек фактически сравнивает цену с самой собой – это и даёт «плотную диагональ». Для товаров со скидкой выполняется `listed_price > total_price`, поэтому точки образуют «крыло» выше линии равенства.

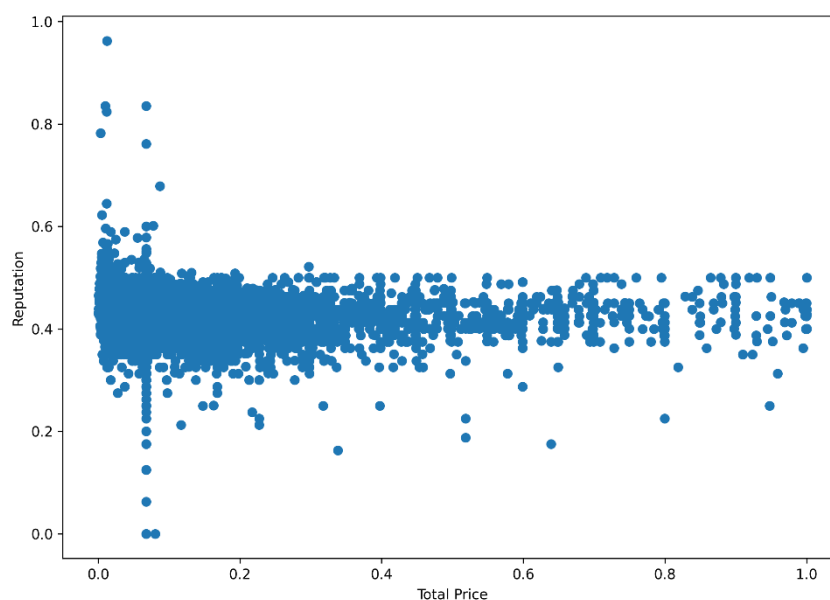


Рисунок 2 – Диаграмма рассеяния reputation по total_price

Распределение (рис. 2) показывает, что высокая цена не гарантирует высокой репутации товара. Большинство товаров сосредоточено в области со средней ценой и средней репутацией.

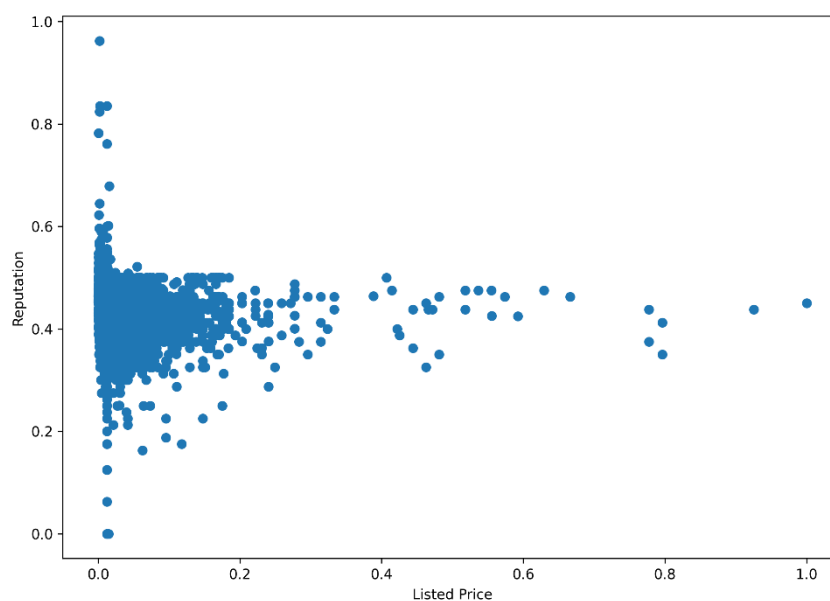


Рисунок 3 – Диаграмма рассеяния reputation по listed_price

Картина аналогична предыдущей: изначальная цена также не оказывает прямого влияния на репутацию (рис. 3). Репутация формируется скорее за счёт отзывов покупателей, чем за счёт стоимости.

Диаграммы рассеяния использовались для выявления возможных зависимостей между ценами и репутацией. Анализ показал, что между

listed_price и total_price наблюдается сильная корреляция, а вот связи между ценами и репутацией почти нет.

2.2. Ящики с усами

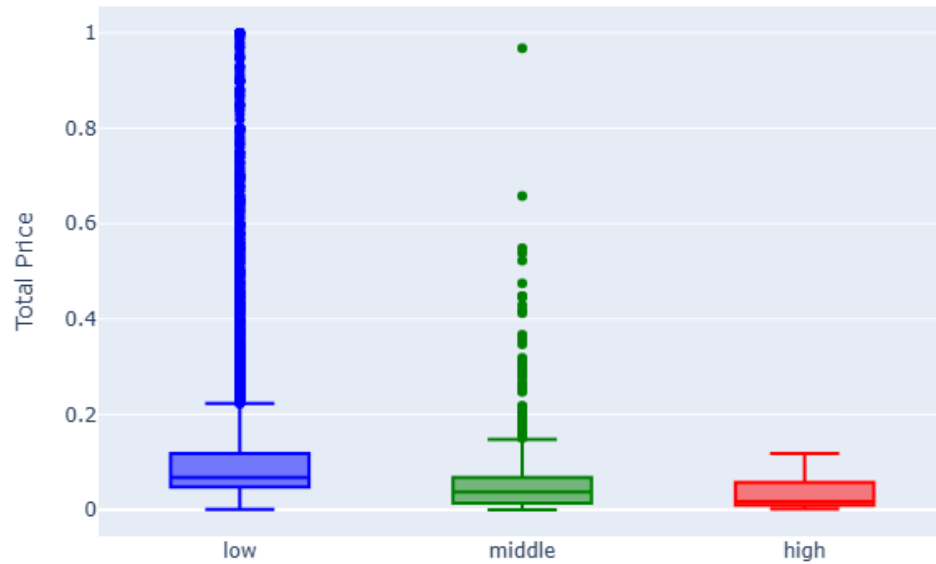


Рисунок 4 – Ящики с усами total_price по demand

На рисунке 4 видно, что товары с высоким спросом чаще имеют низкую или среднюю цену, тогда как дорогие товары попадают в группы с низким спросом.

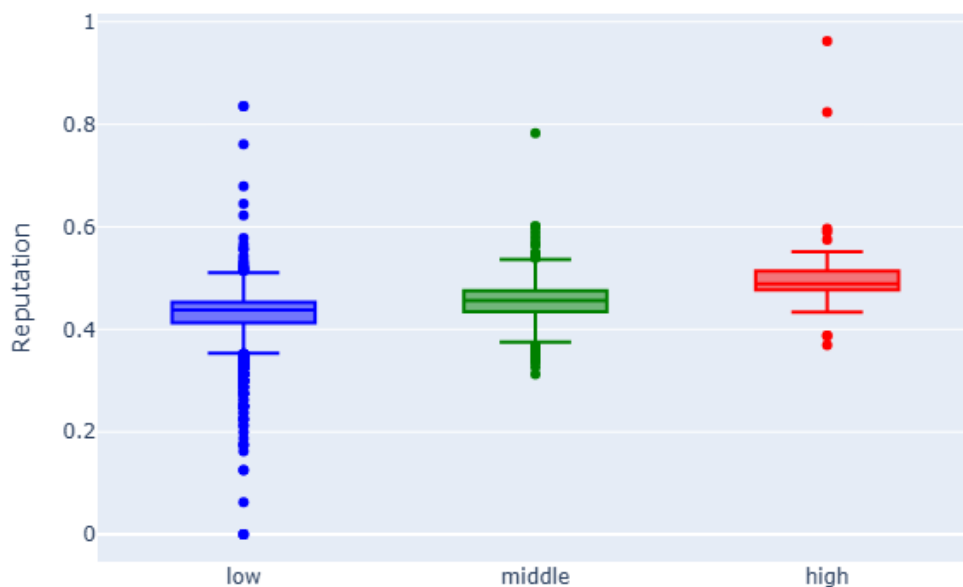


Рисунок 5 – Ящики с усами reputation по demand

На рисунке 5 видно, что репутация выше у товаров с большим спросом. Это логично, так как они получают больше отзывов и чаще выбираются покупателями.

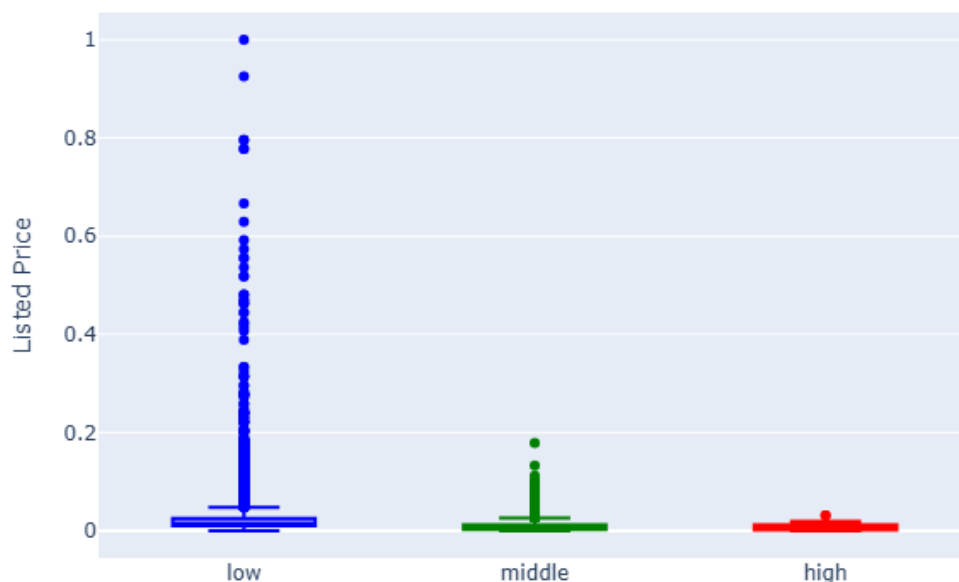


Рисунок 6 – Ящики с усами listed_price по demand

На рисунке 6 ещё раз видно, что товары с низким спросом в среднем дороже. Для высоко востребованных товаров характерна более низкая цена.

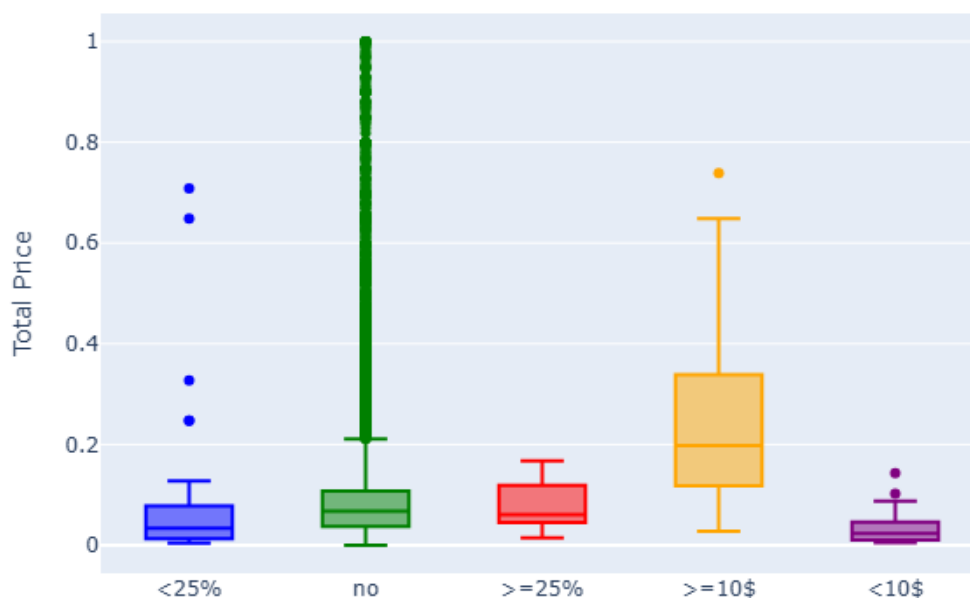


Рисунок 7 – Ящики с усами total_price по is_couponed

На рисунке 7 можно заметить, что товары с купонами чаще находятся в среднем ценовом сегменте. Сильно дорогие товары реже сопровождаются скидками.

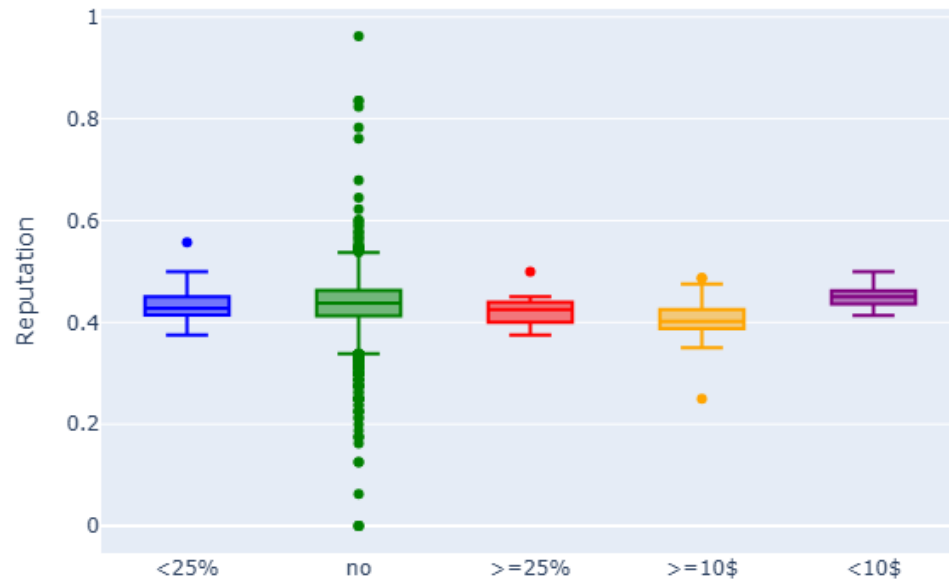


Рисунок 8 – Ящики с усами reputation по is_couponed

Из данных на рисунке 8 можно сделать вывод, что наличие купонов не оказывает заметного влияния на репутацию товара.

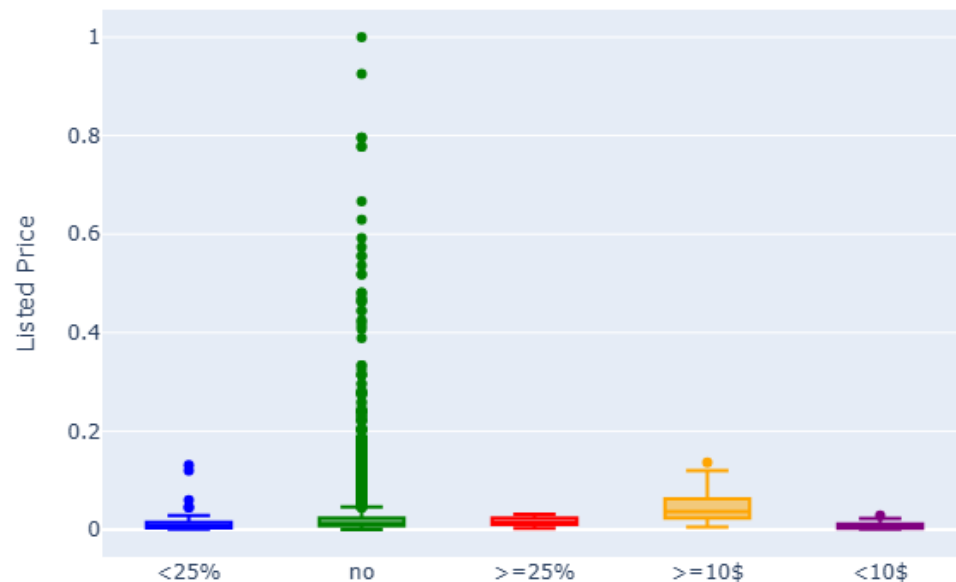


Рисунок 9 – Ящики с усами listed_price по is_couponed

На рисунке 9 распределение схоже с total_price: купоны чаще встречаются у товаров со средней стоимостью.

Ящики с усами использовались для анализа распределений цен и репутации в разрезе спроса и купонов. Диаграммы подтвердили гипотезу, что высокий спрос чаще связан с более низкими ценами и хорошей репутацией, а купоны в основном применяются к товарам среднего ценового сегмента.

2.3. Гистограммы

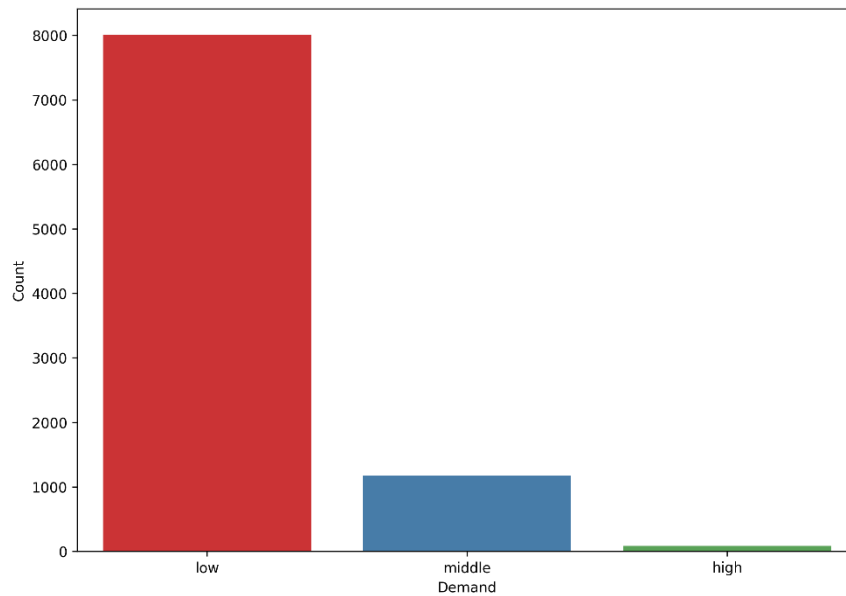


Рисунок 10 – Гистограмма по demand

На рисунке 10 хорошо видно, что распределение резко несбалансированно. Это следствие реального «длинного хвоста» продаж.

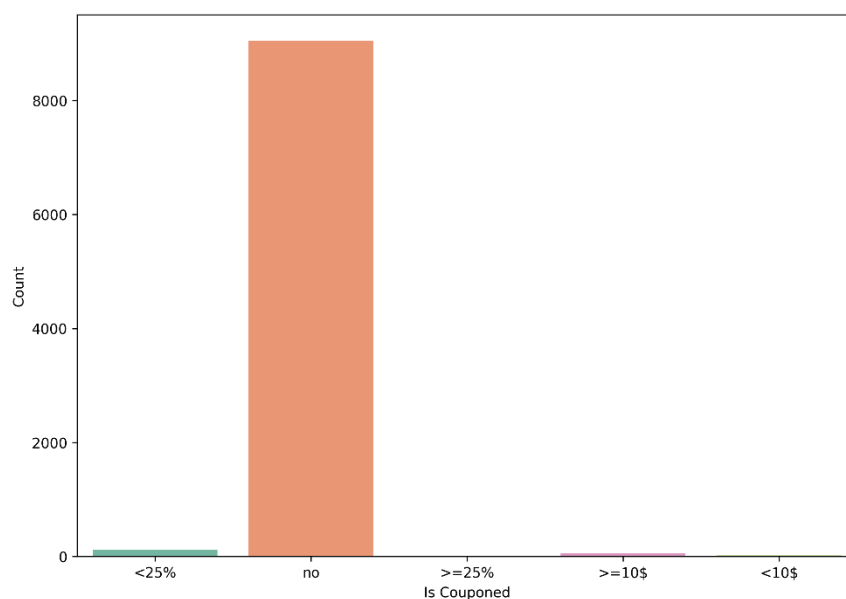


Рисунок 11 – Гистограмма по is_couponed

На рисунке 11 видно, что почти все товары без купонов. Остальные категории составляют очень маленькие доли.

Гистограммы использованы для оценки дисбаланса классов. И спрос, и купоны распределены крайне неравномерно.

3. СТАТИСТИЧЕСКИЙ АНАЛИЗ

Для анализа распределений и взаимосвязей между признаками использовались методы статистики и визуализации. В первую очередь была построена первичная матрица диаграмм рассеяния.

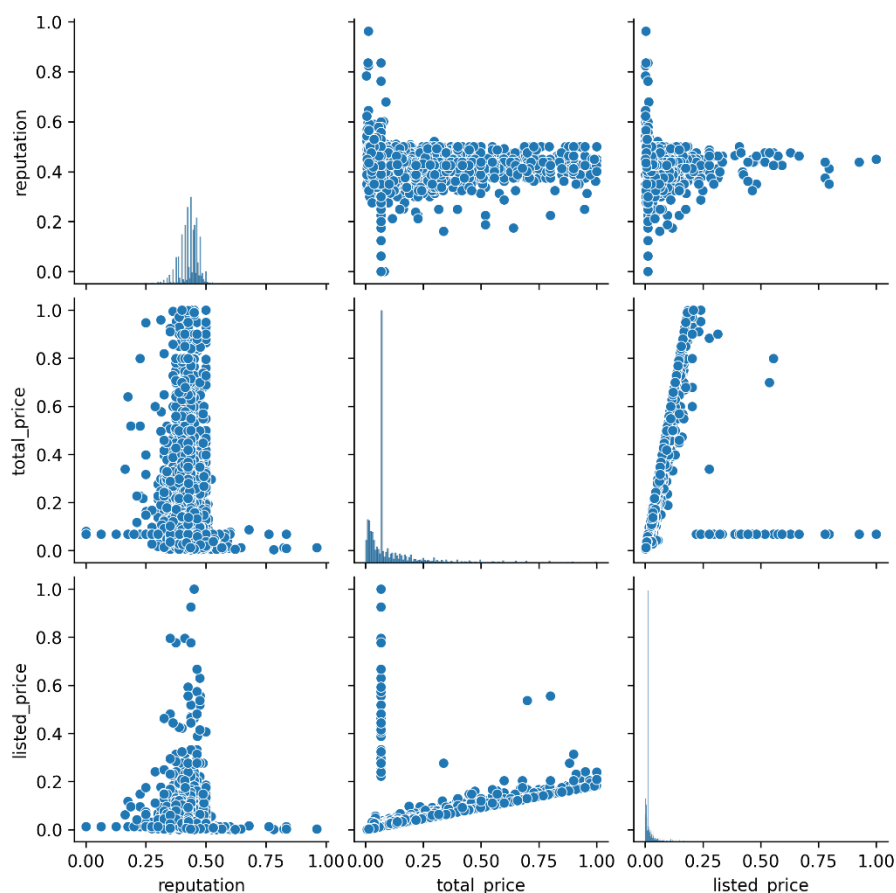


Рисунок 12 – Первичная матрица

На общем рисунке 12 наблюдаются:

- Сильная линейная зависимость `listed_price` и `total_price`.
- Для признака `reputation` характерно сосредоточение значений в узком диапазоне.
- Между `reputation` и ценами нет выраженной корреляции: облака точек распределены равномерно, без наклона.

Для исключения выбросов был применён метод «трёх сигм». Метод основан на свойстве нормального распределения: около 99,7% значений находятся в пределах 3 стандартных отклонений от среднего. Следовательно, значения, выходящие за эти пределы, можно рассматривать как аномальные

выбросы. Такой подход прост в реализации, не требует дополнительных предположений и хорошо подходит для больших выборок по типу нашей.

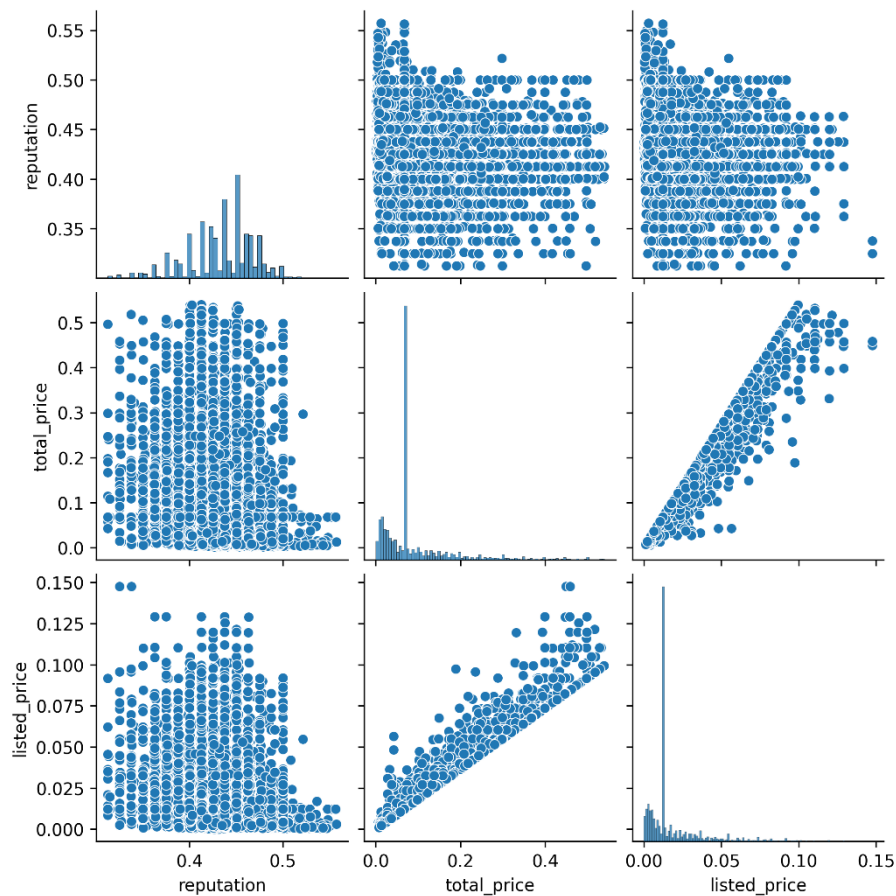


Рисунок 13 – Матрица без выбросов

После удаления выбросов методом «трёх сигм» (рис. 13) картина стала чище:

- Линейная зависимость между `listed_price` и `total_price` ещё более явно прослеживается.
- Шумные выбросы в области высоких цен исчезли, что улучшило читаемость графиков.
- Распределение `reputation` стало более плотным, что позволяет корректнее сравнивать её с другими признаками.

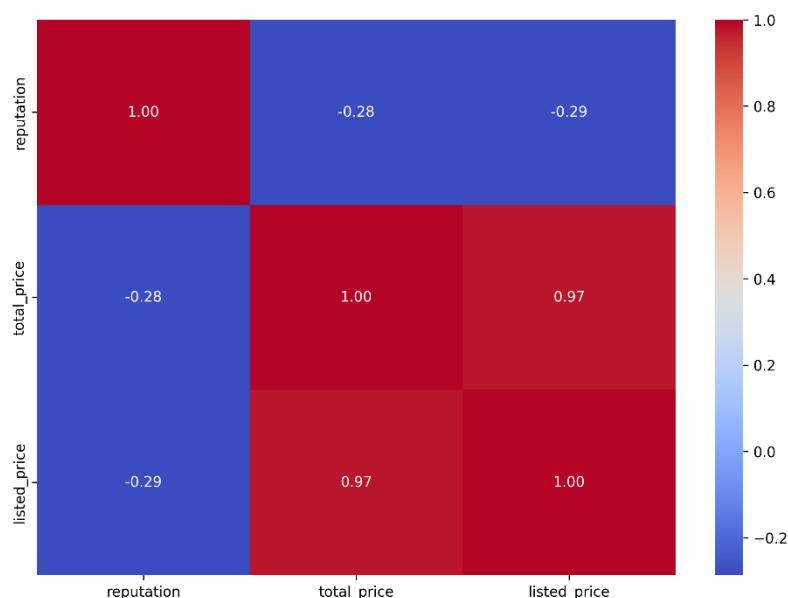


Рисунок 14 - Корреляционная матрица

На тепловой карте корреляций (рис. 14) видно:

- Между `listed_price` и `total_price` коэффициент Спирмена равен 0.97, что указывает на очень сильную зависимость. Это объясняется тем, что `total_price` формируется на основе `listed_price` с учётом скидок.
- Связь между `reputation` и ценовыми признаками отрицательная и слабая (-0.28 и -0.29). Это подтверждает сделанный ранее вывод: высокая цена не гарантирует высокой репутации товара.

Таким образом, цена и «репутация» отражают разные стороны данных: первая связана с товарным позиционированием, вторая – с пользовательской активностью и отзывами.

ВЫВОДЫ

В ходе выполнения лабораторной работы была проведена полная предобработка и анализ большого датасета с данными о товарах Amazon.

Предобработка данных:

- Выполнено преобразование строковых признаков в числовые, нормализация числовых данных, обработка пропусков и удаление дубликатов.
- Созданы новые информативные признаки (*reputation*, *demand*), что позволило получить более содержательные результаты анализа.
- После очистки осталось 9262 уникальных записи, чего достаточно для корректной статистической обработки.

Визуализация признаков:

- Диаграммы рассеяния показали, что между *listed_price* и *total_price* существует сильная зависимость, обусловленная тем, что текущая цена формируется на основе исходной. При этом *reputation* не связана напрямую с ценой, а отражает скорее активность покупателей и их удовлетворённость.
- Ящики с усами позволили выявить различия в распределении цен и репутации в зависимости от спроса и наличия купонов. Было подтверждено, что высокий спрос характерен для товаров более дешёвого сегмента и с более высокой репутацией. Купоны чаще встречаются у товаров средней цены, но не оказывают значимого влияния на репутацию.
- Гистограммы выявили сильный дисбаланс категориальных признаков: подавляющее большинство товаров имеет низкий спрос и не сопровождается купонами.

Статистический анализ:

- Метод «трёх сигм» был использован для выявления и удаления выбросов. Это позволило очистить данные и улучшить интерпретацию диаграмм рассеяния и матрицы корреляций.

- Корреляционный анализ (по Спирмену) подтвердил почти полную зависимость между *listed_price* и *total_price* и слабую отрицательную зависимость цен и *reputation*.

Таким образом, проведённая работа позволила на практике освоить полный цикл предобработки данных, визуального и статистического анализа. Полученные выводы могут быть полезны при построении моделей машинного обучения для прогнозирования спроса и рейтингов товаров.

ПРИЛОЖЕНИЕ

Код программы

```
import pandas as pd
import numpy as np
pd.plotting.register_matplotlib_converters()

filepath = "data/amazon products sales data uncleaned.csv"
data = pd.read_csv(filepath)

data["title"] = data["title"].str.lower()

data["rating"] = (
    data["rating"]
    .str.extract(r"(\d+(?:\.\d+)?)")
    .astype(float)
    .round(1)
)
median_val = data["rating"].median()
data["rating"] = data["rating"].fillna(median_val)

data["number of reviews"] = (
    data["number_of_reviews"]
    .astype(str)
    .str.replace(",", "", regex=False)
    .replace("nan", np.nan)
    .astype(float)
)
median_val = data["number of reviews"].median()
data["number of reviews"] = data["number of reviews"].fillna(median_val)
data["number of reviews"] = data["number of reviews"].astype(int)

bought_raw = data["bought in last month"].astype(str)
numbers = bought_raw.str.extract(r"(\d+\.\d*) ([KM]?)") [0]
suffixes = bought_raw.str.extract(r"(\d+\.\d*) ([KM]?)") [1]
data["bought in last month"] = pd.to_numeric(numbers, errors="coerce")
data["bought in last month"] = np.where(
    suffixes == "K", data["bought in last month"] * 1000,
    np.where(suffixes == "M", data["bought in last month"] * 1 000 000,
    data["bought in last month"])
)
median_val = data["bought in last month"].median()
data["bought in last month"] = data["bought in last month"].fillna(median_val).astype(int)

data["current/discounted price"] = pd.to_numeric(data["current/discounted price"],
errors="coerce").round(2)
median_val = data["current/discounted price"].median()
data["current/discounted price"] = data["current/discounted price"].fillna(median_val)
data = data.rename(columns={"current/discounted price": "total price"})

data = data.drop("price on variant", axis=1)

data["listed price"] = data["listed price"].replace("No Discount", np.nan)
data["listed price"] = data["listed price"].str.replace("$", "",
regex=True).astype(float)
data["listed price"] = data["listed price"].fillna(data["total price"]).round(2)

data = data.drop("is best seller", axis=1)

data["is sponsored"] = data["is sponsored"].map(lambda x: 1 if x == "Sponsored" else 0)

coupon = data["is couponed"].astype(str)
mask_no = coupon.str.contains("No Coupon", case=False, na=False)
mask_dollar = coupon.str.contains(r"\$", na=False)
dollar_val = (coupon.str.extract(r"(\d+\.\d*)") [0].astype(float))
mask_dollar_ge10 = mask_dollar & (dollar_val >= 10)
mask_dollar_lt10 = mask_dollar & (dollar_val < 10)
mask_percent = coupon.str.contains(r"%", na=False)
percent_val = (coupon.str.extract(r"(\d+\.\d*)") [0].astype(float))
mask_percent_ge25 = mask_percent & (percent_val >= 25)
mask_percent_lt25 = mask_percent & (percent_val < 25)
data["is couponed"] = np.select(
    [
        mask_no,

```

```

        mask dollar ge10,
        mask dollar lt10,
        mask percent ge25,
        mask percent lt25
    ],
    [
        "no",
        ">=10$",
        "<10$",
        ">=25%",
        "<25%"
    ],
    default="no"
)

data["buy box availability"] = data["buy box availability"].map(lambda x: 1 if x == "Add to
cart" else 0)

data = data.drop(columns=[
    "delivery details",
    "sustainability badges",
    "image url",
    "product url",
    "collected at"
])

data.to_csv("data/amazon products sales data full.csv", index=False)

from sklearn.preprocessing import MinMaxScaler

numeric_cols = [
    "rating",
    "number of reviews",
    "bought in last month",
    "total price",
    "listed price"
]

scaler = MinMaxScaler()
data[numeric_cols] = scaler.fit_transform(data[numeric_cols])

data["reputation"] = data[["rating", "number of reviews"]].mean(axis=1)
data = data.drop(columns=["rating", "number_of_reviews"])

def demand_group(x):
    if 0 <= x <= 0.01:
        return "low"
    elif 0.01 < x <= 0.1:
        return "middle"
    else:
        return "high"

data["demand"] = data["bought in last month"].apply(demand_group)
data = data.drop(columns=["bought_in_last_month"])

data.to_csv("data/amazon products sales data normalized.csv", index=False)

data = data.drop_duplicates()

data.to_csv("data/amazon products sales data cleaned.csv", index=False)

import matplotlib.pyplot as plt
import seaborn as sns
import plotly.graph_objects as go

plt.figure(figsize=(10,7))
plt.scatter('total price', 'listed price', data=data)
plt.xlabel("Total Price")
plt.ylabel("Listed Price")
plt.savefig("images/scatter total vs listed.png", dpi=300, bbox_inches="tight")
plt.close()

plt.figure(figsize=(10,7))
plt.scatter('total price', 'reputation', data=data)
plt.xlabel("Total Price")

```

```

plt.ylabel("Reputation")
plt.savefig("images/scatter total vs reputation.png", dpi=300, bbox_inches="tight")
plt.close()

plt.figure(figsize=(10,7))
plt.scatter('listed price', 'reputation', data=data)
plt.xlabel("Listed Price")
plt.ylabel("Reputation")
plt.savefig("images/scatter listed vs reputation.png", dpi=300, bbox_inches="tight")
plt.close()

fig = go.Figure()
for group, color in zip(["low", "middle", "high"], ["blue", "green", "red"]):
    fig.add_trace(go.Box(
        y=data.loc[data["demand"] == group, "total price"],
        name=group,
        marker_color=color
    ))
fig.update_layout(title="Boxplot: Total Price no Demand", yaxis_title="Total Price")
fig.write_image("images/box total price by demand.png")

fig = go.Figure()
for group, color in zip(["low", "middle", "high"], ["blue", "green", "red"]):
    fig.add_trace(go.Box(
        y=data.loc[data["demand"] == group, "reputation"],
        name=group,
        marker_color=color
    ))
fig.update_layout(title="Boxplot: Reputation no Demand", yaxis_title="Reputation")
fig.write_image("images/box reputation by demand.png")

fig = go.Figure()
for group, color in zip(["low", "middle", "high"], ["blue", "green", "red"]):
    fig.add_trace(go.Box(
        y=data.loc[data["demand"] == group, "listed price"],
        name=group,
        marker_color=color
    ))
fig.update_layout(title="Boxplot: Listed Price no Demand", yaxis_title="Listed Price")
fig.write_image("images/box listed price by demand.png")

fig = go.Figure()
for group, color in zip(data["is_couped"].unique(), ["blue", "green", "red", "orange", "purple"]):
    fig.add_trace(go.Box(
        y=data.loc[data["is_couped"] == group, "total price"],
        name=group,
        marker_color=color
    ))
fig.update_layout(title="Boxplot: Total Price no Is Couped", yaxis_title="Total Price")
fig.write_image("images/box total price by couped.png")

fig = go.Figure()
for group, color in zip(data["is_couped"].unique(), ["blue", "green", "red", "orange", "purple"]):
    fig.add_trace(go.Box(
        y=data.loc[data["is_couped"] == group, "reputation"],
        name=group,
        marker_color=color
    ))
fig.update_layout(title="Boxplot: Reputation no Is Couped", yaxis_title="Reputation")
fig.write_image("images/box reputation by couped.png")

fig = go.Figure()
for group, color in zip(data["is_couped"].unique(), ["blue", "green", "red", "orange", "purple"]):
    fig.add_trace(go.Box(
        y=data.loc[data["is_couped"] == group, "listed price"],
        name=group,
        marker_color=color
    ))
fig.update_layout(title="Boxplot: Listed Price no Is Couped", yaxis_title="Listed Price")
fig.write_image("images/box listed price by couped.png")

plt.figure(figsize=(10, 7))
sns.countplot(data=data, x="demand", hue="demand", palette="Set1", order=["low", "middle", "high"])

```

```

plt.xlabel("Demand")
plt.ylabel("Count")
plt.savefig("images/hist demand.png", dpi=300, bbox_inches="tight")
plt.close()

plt.figure(figsize=(10, 7))
sns.countplot(data=data, x="is couponed", hue="is couponed", palette="Set2",
order=data["is couponed"].unique())
plt.xlabel("Is Couponed")
plt.ylabel("Count")
plt.savefig("images/hist is couponed.png", dpi=300, bbox_inches="tight")
plt.close()

numeric_cols = [
    "reputation",
    "total price",
    "listed price"
]

sns.pairplot(data[numeric_cols])
plt.savefig("images/pairplot all.png", dpi=300, bbox_inches="tight")
plt.close()

def outliers_indices(df, feature):
    mean = df[feature].mean()
    std = df[feature].std()
    return df[(df[feature] < mean - 3*std) | (df[feature] > mean + 3*std)].index
outliers_sets = [outliers_indices(data, col) for col in numeric_cols]
outliers_all = set().union(*outliers_sets)
data_clean = data.drop(outliers_all)

sns.pairplot(data_clean[numeric_cols])
plt.savefig("images/pairplot clean.png", dpi=300, bbox_inches="tight")
plt.close()

plt.figure(figsize=(10, 7))
sns.heatmap(data_clean[numeric_cols].corr(method='spearman'), annot=True, fmt=".2f",
cmap="coolwarm")
plt.savefig("images/heatmap corr.png", dpi=300, bbox_inches="tight")
plt.close()

```